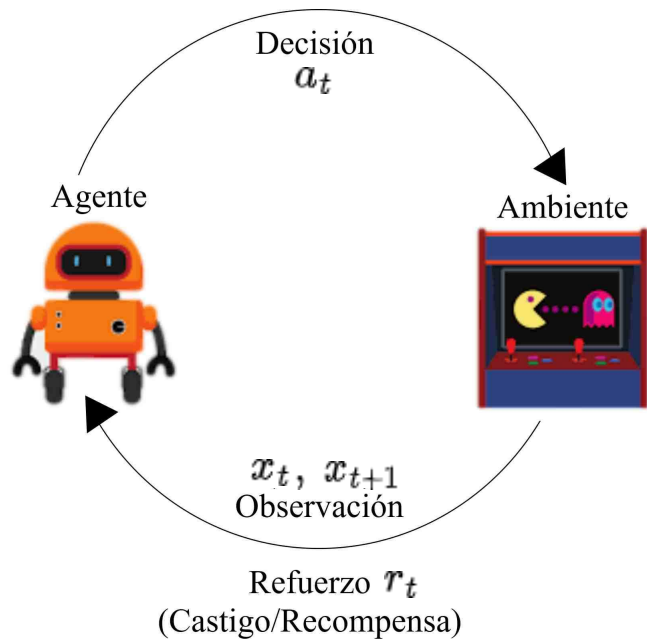


Aprendizaje por Refuerzo - Aproximaciones

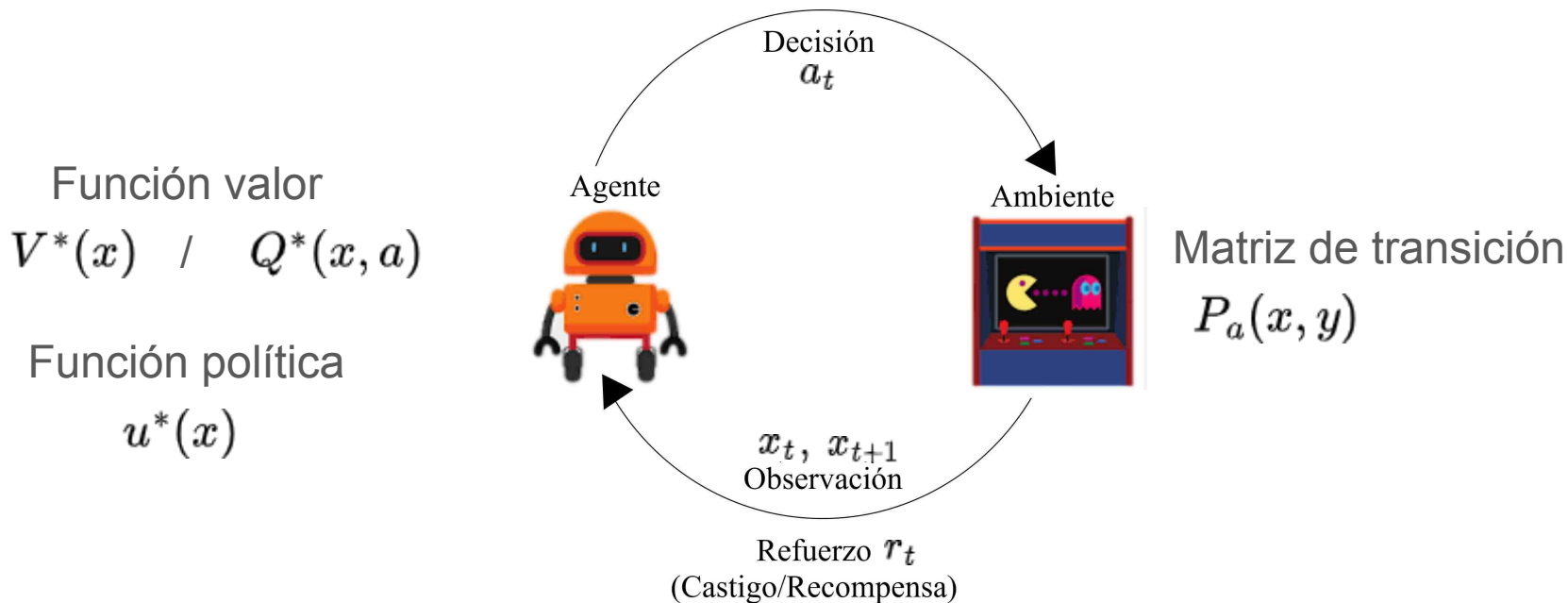
Clase 5

- La “Maldición de la Dimensionalidad”
 - MDPs de larga escala
 - Arquitecturas de aproximación
- Aplicación: Precificación de opciones
 - Teoría de opciones
 - Formulación MDP y modelaje de precios de activos
 - Simulación Monte Carlo

Revisión - MDPs y Aprendizaje por Refuerzo



Revisión - MDPs y Aprendizaje por Refuerzo



Revisión - MDPs y Aprendizaje por Refuerzo

- Función valor definida por la ecuación de Bellman

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a')$$

Revisión - MDPs y Aprendizaje por Refuerzo

- Función valor definida por la ecuación de Bellman

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a')$$

- Resolvemos la ecuación de Bellman exactamente o con aprendizaje:

Soluciones Exactas	Aprendizaje
Iteración de valor Iteración de política	Simulación Monte Carlo (política fija) TD-Learning (política fija o variable) Q-Learning

Revisión - MDPs y Aprendizaje por Refuerzo

- Función valor definida por la ecuación de Bellman

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a')$$

- Resolvemos la ecuación de Bellman exactamente o con aprendizaje:

Soluciones Exactas	Aprendizaje
Iteración de valor Iteración de política	Simulación Monte Carlo (política fija) TD-Learning (política fija o variable) Q-Learning

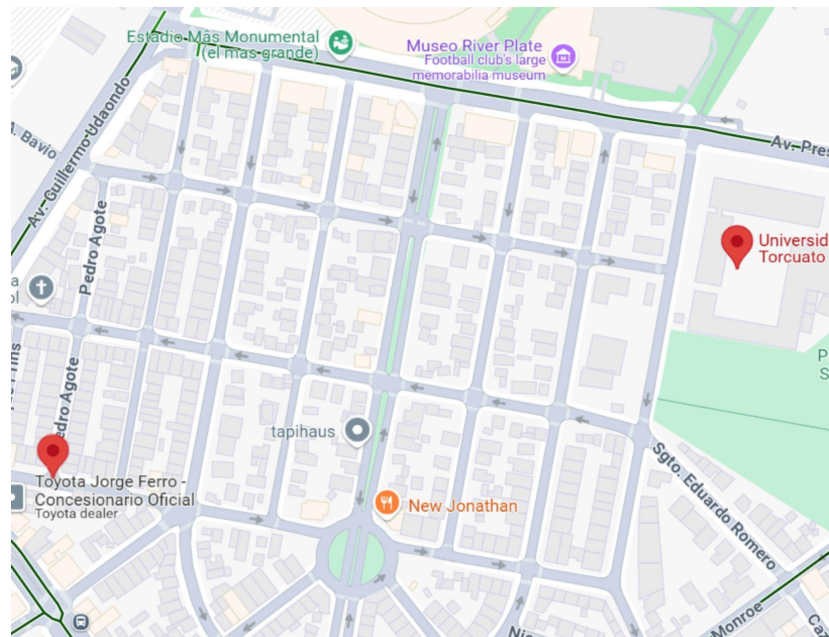
- Calculamos la política óptima a partir de $V^*(x)$ o $Q^*(x, a)$.

La “Maldición de La Dimensionalidad”

- Debemos almacenar $V^*(x)$ o $Q^*(x, a)$.
- Tabla con un valor por estado o par estado/acción.
- Cuánto espacio necesario?

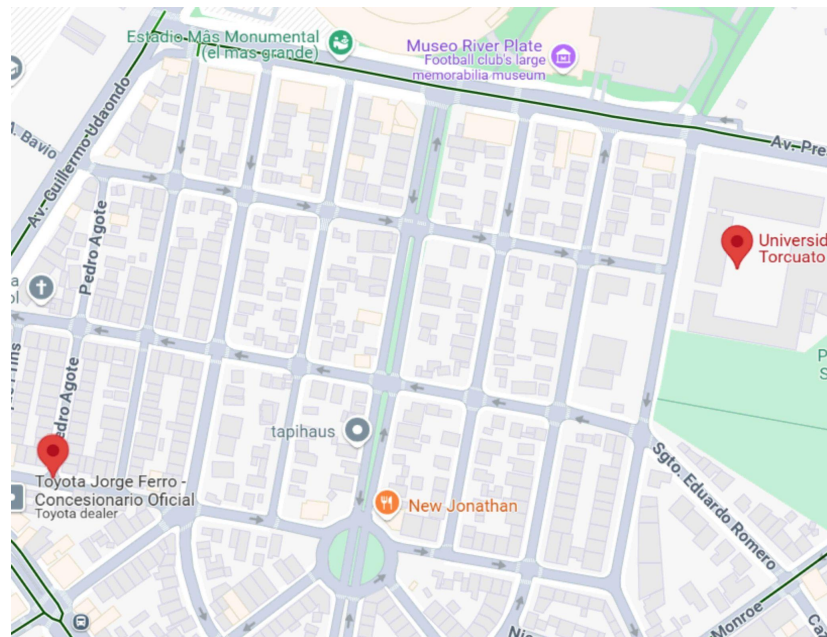
La “Maldición de La Dimensionalidad”

- Debemos almacenar $V^*(x)$ o $Q^*(x, a)$.
- Tabla con un valor por estado o par estado/acción.
- Cuánto espacio necesario?
- Ejemplo: Ruta casa a trabajo
 - N calles, M columnas

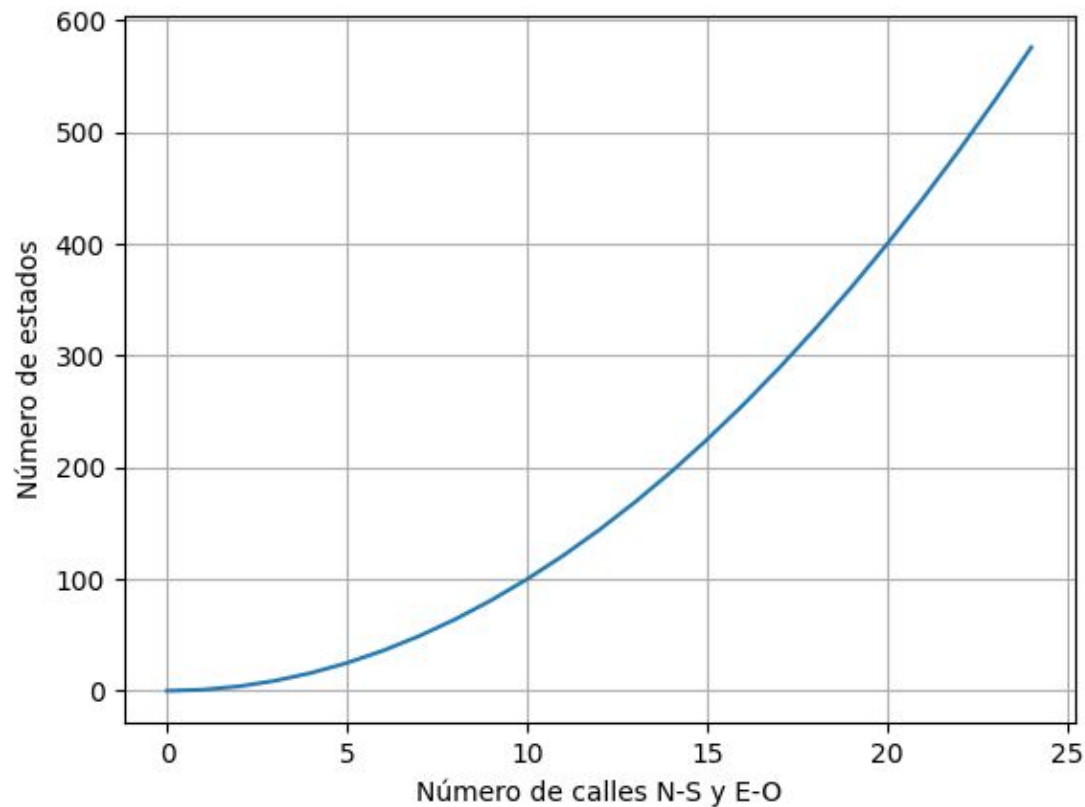


La “Maldición de La Dimensionalidad”

- Debemos almacenar $V^*(x)$ o $Q^*(x, a)$.
- Tabla con un valor por estado o par estado/acción.
- Cuánto espacio necesario?
- Ejemplo: Ruta casa a trabajo
 - N calles, M columnas
 - Número de estados = $N \times M$
 - Número de estados/acciones = $N \times M \times 4$



La “Maldición de La Dimensionalidad”



La “Maldición de La Dimensionalidad”

- Número de estados = $N \times M$
 - Crecimiento cuadrático en el número de estados con las dimensiones del grid

La “Maldición de La Dimensionalidad”

- Número de estados = $N \times M$
 - Crecimiento cuadrático en el número de estados con las dimensiones del grid
 - No tan dramático, porque son pocas dimensiones (2 para el estado, 1 para las acciones)
 - Variable de estado = (i,j)
 - Variable de acción en $\{N, S, E, O\}$

La “Maldición de La Dimensionalidad”

- Número de estados = $N \times M$
 - Crecimiento cuadrático en el número de estados con las dimensiones del grid
 - No tan dramático, porque son pocas dimensiones (2 para el estado, 1 para las acciones)
 - Variable de estado = (i,j)
 - Variable de acción en $\{N, S, E, O\}$
- Nuevo problema: a cada día, una de las calles en el grid puede estar cortada
 - Cómo modelar ese problema?
 - Cuántos estados?

La “Maldición de La Dimensionalidad”

- Nuevo problema: a cada día, una de las calles en el grid puede estar cortada
 - Cómo modelar ese problema?
 - Cuántos estados?
- Cada calle c tiene una variable de estado con valores $\{?, \text{libre}, \text{cortada}\}$
 - Estados posibles:
 - Si aún no hemos encontrado una calle cortada:
 - $(c_1, c_2, \dots, c_n), c_i \in \{?, \text{libre}\}$
 - 2^n posibilidades
 - Si hemos encontrado una calle cortada:
 - $(c_1, c_2, \dots, c_n) = (\text{libre}, \text{libre}, \dots, \text{libre}, \text{cortada}, \text{libre}, \dots, \text{libre})$
porque suponemos que hay solo una calle cortada
 - n posibilidades

La “Maldición de La Dimensionalidad”

- Nuevo problema: a cada día, una de las calles en el grid puede estar cortada
 - Cómo modelar ese problema?
 - Cuántos estados?
- Cada calle c tiene una variable de estado con valores $\{?, \text{libre}, \text{cortada}\}$
 - Estados posibles:
 - Si aún no hemos encontrado una calle cortada:
 - $(c_1, c_2, \dots, c_n), c_i \in \{?, \text{libre}\}$
 - 2^n posibilidades
 - Si hemos encontrado una calle cortada:
 - $(c_1, c_2, \dots, c_n) = (\text{libre}, \text{libre}, \dots, \text{libre}, \text{cortada}, \text{libre}, \dots, \text{libre})$
porque suponemos que hay solo una calle cortada
 - n posibilidades
- **Número de estados crece exponencialmente con el número de variables de estado - > no podemos almacenar la función valor**

Ejemplo: Go



Ejemplo: Go

Configuración y fuerza⁷

Versiones	Hardware	Elo	Partidos
AlphaGo Fan	176 GPUs, distribuido	3.144	5:0 contra Fan Hui
AlphaGo Lee	48 TPUs, distribuido	3.739	4:1 contra Lee Sedol
AlphaGo Master	Una sola máquina con 4 TPU v2	4.858	60:0 contra jugadores profesionales; Cumbre del Futuro de Go
AlphaGo Zero	Una sola máquina con 4 TPUs ⁸ v2	5.185 ⁹	100:0 contra AlphaGo Lee 89:11 contra AlphaGo Master

Ejemplo: Go

- Cuántos estados?
- Tablero de 19x19
 - $19 \times 19 = 361$ posiciones/variables de estado
 - cada posición puede tener 3 valores: blanco, negro, vacío
 - 3^{361} estados

Ejemplo: Go

- Cuántos estados?
- Tablero de 19x19
 - 19 x 19 = 361 posiciones/variables de estado
 - cada posición puede tener 3 valores: blanco, negro, vacío
 - 3^{361} estados
 - pero no todas las configuraciones son válidas
 - aproximadamente $2.081681994 * 10^{170}$ posiciones (2006)

Ejemplo: Go

- Cuántos estados?
- Tablero de 19x19
 - $19 \times 19 = 361$ posiciones/variables de estado
 - cada posición puede tener 3 valores: blanco, negro, vacío
 - 3^{361} estados
 - pero no todas las configuraciones son válidas
 - aproximadamente $2.081681994 \times 10^{170}$ posiciones (2006)
 - en 2015, con avances en la potencia de computadores y nueve meses de procesamiento, John Tromp calculó el número de estados:

2081681993819799846
9947863334486277028
6522453884530548425
6394568209274196127
3801537852564845169
8519643907259916015
6281285460898883144
2712971531931755773
6620397247064840935

La “Maldición de La Dimensionalidad”

- En la práctica, número de estados es frecuentemente prohibitivo. No podemos calcular y almacenar la función valor.
- Solución: **generalización**
- Aproximamos la función valor y/o la función política con una forma paramétrica:

$$V^*(x) \approx f(x, w), \quad w \in \mathfrak{R}^K$$

$$Q^*(x, a) \approx f(x, a, w), \quad w \in \mathfrak{R}^K$$

$$u_a^*(x) \approx f(x, a, w), \quad w \in \mathfrak{R}^K$$

Arquitecturas de Aproximación Lineales

$$V^*(x) \approx f(x, w) = \sum_{k=1}^K w_k \phi_k(x)$$

- Arquitectura lineal en w_k
- $\phi_k(x)$ llamadas *funciones básicas*
- Funciones lineales en x :

$$\phi_k(x) = x_k$$

- Polinomios, sinusoides, etc.

Arquitecturas de Aproximación Lineales

$$V^*(x) \approx f(x, w) = \sum_{k=1}^K w_k \phi_k(x)$$

- Arquitectura lineal en w_k
- $\phi_k(x)$ llamadas *funciones básicas*
- Agregamiento de estados:

$$\phi_k(x) = \begin{cases} 1 & \text{if } x \in S_k \\ 0 & \text{otherwise} \end{cases}$$

Arquitecturas de Aproximación Lineales

Agregamiento de Estados

- Agregamiento de estados:

$$\phi_k(x) = \begin{cases} 1 & \text{if } x \in S_k \\ 0 & \text{otherwise} \end{cases}$$

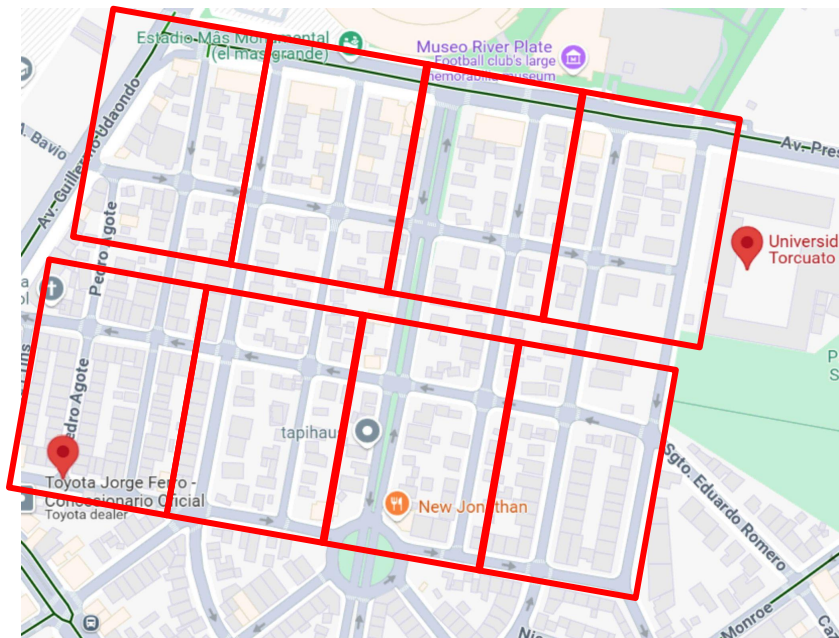
Arquitecturas de Aproximación Lineales

Agregamiento de Estados

- Agregamiento de estados:

$$\phi_k(x) = \begin{cases} 1 & \text{if } x \in S_k \\ 0 & \text{otherwise} \end{cases}$$

$$V^*((i, j)) \approx V(\text{zona}(i, j))$$



Arquitecturas de Aproximación Lineales

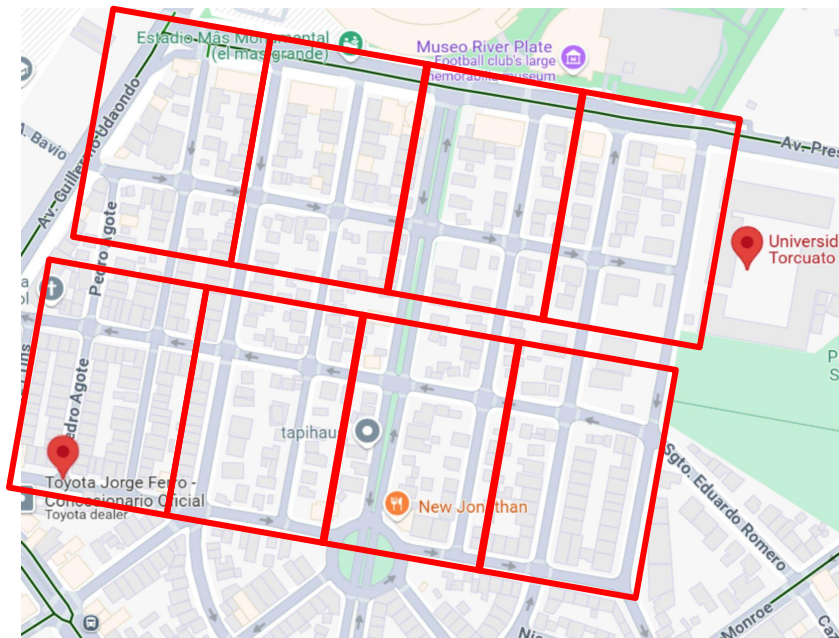
Agregamiento de Estados

- Agregamiento de estados:

$$\phi_k(x) = \begin{cases} 1 & \text{if } x \in S_k \\ 0 & \text{otherwise} \end{cases}$$

$$V^*((i, j)) \approx V(\text{zona}(i, j))$$

- Buenas propiedades de convergencia
- Posibilidad de particionar espacio de estados progresivamente



Arquitecturas de Aproximación No Lineales

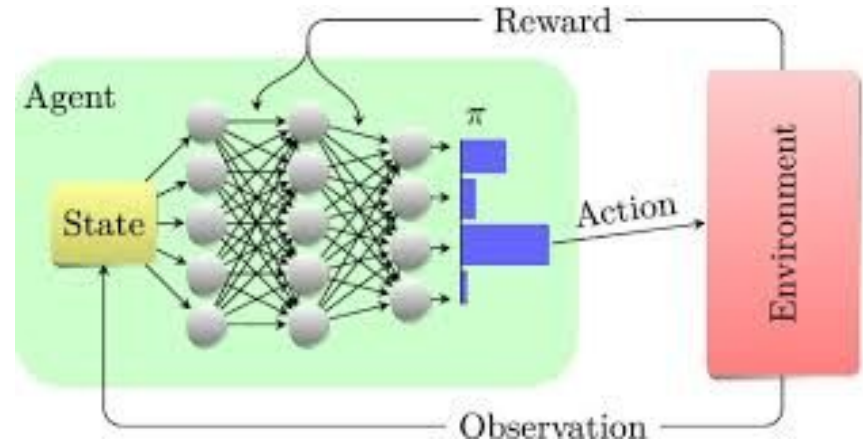
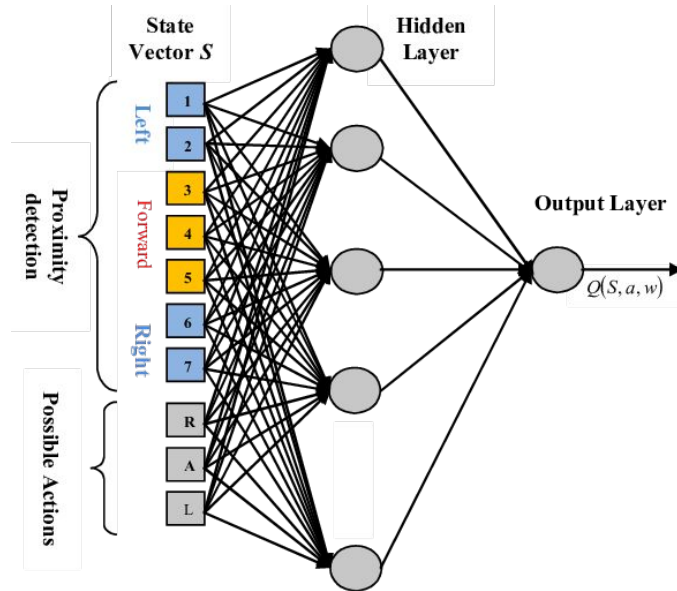
$$V^*(x) \approx f(x, w), \quad w \in \mathfrak{R}^K$$

- f es no lineal en w

Arquitecturas de Aproximación No Lineales

$$V^*(x) \approx f(x, w), w \in \mathfrak{R}^K$$

- f es no lineal en w
- redes neuronales



Aprendizaje por Refuerzo con Aproximaciones

- Recordemos que queremos resolver la ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- Con aproximación $V^*(x) \approx f(x, w)$, $w \in \mathfrak{R}^K$, algoritmos resuelven

$$\min_w \sum_x \underbrace{p(x)}_{\text{peso del estado } x} \underbrace{\left[\max_{a \in A} \left(r_a(x) + \alpha \sum_y P_a(x, y) f(y, w) \right) - f(x, w) \right]^2}_{\text{error de Bellman}}$$

Aprendizaje por Refuerzo con Aproximaciones

$$\min_w \sum_x p(x) \left[\max_{a \in A} \left(r_a(x) + \alpha \sum_y P_a(x, y) f(y, w) \right) - f(x, w) \right]^2$$

- Cómo resolver?
- Algoritmos basados en simulaciones o aprendizaje
 - gradiente estocástico
 - exploración vs explotación
- Convergencia no siempre asegurada, muchas técnicas y experimentos necesarios
- Con redes neuronales: DQN (Deep Q Networks)

Aplicación: Valuación de Opciones Americanas

- Opciones son instrumentos derivados que otorgan el derecho -y no la obligación- a comprar o vender activos a un precio determinado en el contrato -denominado precio de ejercicio o strike- hasta una fecha establecida, es decir el vencimiento del contrato.
- Derecho a comprar el activo = opción call
- Derecho a vender el activo = opción put

Aplicación: Valuación de Opciones Americanas

- Opciones son instrumentos derivados que otorgan el derecho -y no la obligación- a comprar o vender activos a un precio determinado en el contrato -denominado precio de ejercicio o strike- hasta una fecha establecida, es decir el vencimiento del contrato.
- Derecho a comprar el activo = opción call
- Derecho a vender el activo = opción put
- Ejercicio del derecho solo en la fecha de vencimiento = opción europea
- Ejercicio del derecho hasta la fecha de vencimiento = opción americana

Aplicación: Valuación de Opciones Americanas

- Consideramos una opción put americana, con fecha de vencimiento T y precio de ejercicio K .
- Cuál debe ser el precio de la opción en $t = 0$?
- Cuándo se debe ejercer la opción?

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- Recompensa $r_a(s)$:
 - $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- Recompensa $r_a(s)$:
 - $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)
 - Cuánto ganamos si ejercemos la opción en en tiempo t ?

Modelo MDP para Valuación de Opciones

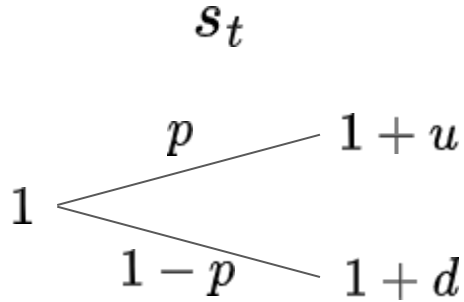
- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- Recompensa $r_a(s)$:
 - $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)
 - Cuánto ganamos si ejercemos la opción en en tiempo t ?
 - Compramos la acción a precio s_t = precio del activo
 - Vendemos a precio K
 - Ganamos $K - s_t$
 - Solo hace sentido ejercer la opción si $K - s_t > 0$

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- Recompensa $r_a(s)$:
 - $r_0(s) = 0$ (si no ejercemos, no hay recompensa inmediata)
 - Cuánto ganamos si ejercemos la opción en tiempo t ?
 - Compramos la acción a precio s_t = precio del activo
 - Vendemos a precio K
 - Ganamos $K - s_t$
 - Solo hace sentido ejercer la opción si $K - s_t > 0$
 - $r_1(s) = \max(K - s, 0)$
- Debemos especificar P , o más generalmente la evolución de precio del activo para terminar de definir el problema...

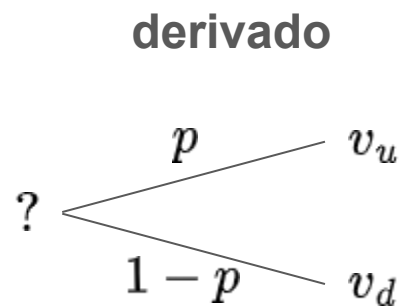
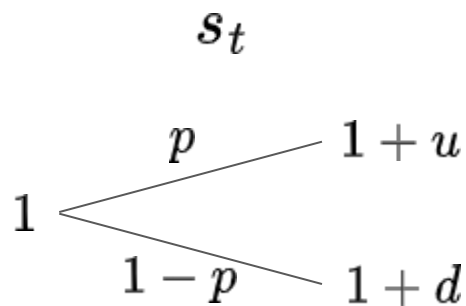
Modelo para s_t

- Consideramos el valor de un derivado del activo con un modelo binomial. Asumimos una tasa de interés sin riesgo r (\$1 en $t = 0$ vale \$ $(1 + r)$ en $t = 1$).



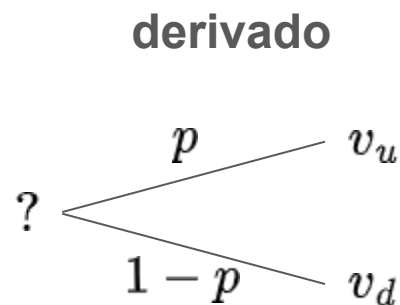
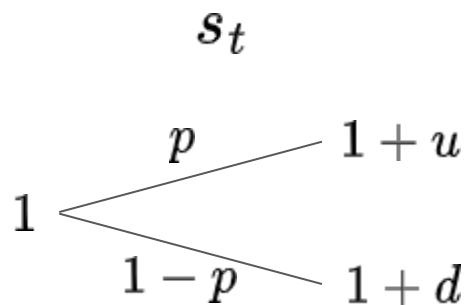
Modelo para s_t

- Consideramos el valor de un derivado del activo con un modelo binomial. Asumimos una tasa de interés sin riesgo r (\$1 en $t = 0$ vale \$ $(1 + r)$ en $t = 1$).



Modelo para s_t

- Consideramos el valor de un derivado del activo con un modelo binomial. Asumimos una tasa de interés sin riesgo r (\$1 en $t = 0$ vale \$ $(1 + r)$ en $t = 1$).

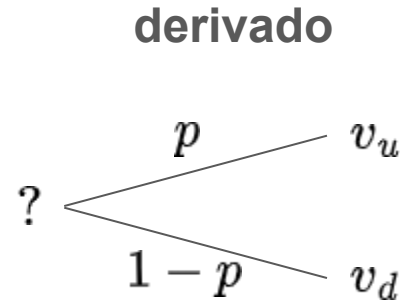
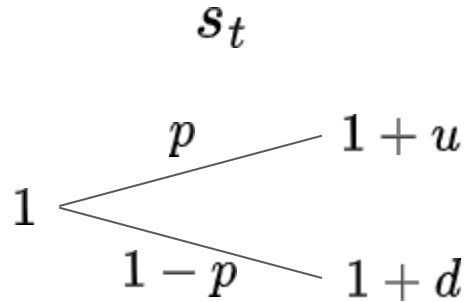


- Podemos calcular el valor presente:

$$? = \frac{p \times v_u + (1 - p) \times v_d}{1 + r}$$

Modelo para s_t

- Consideramos el valor de un derivado del activo con un modelo binomial. Asumimos una tasa de interés sin riesgo r (\$1 en $t = 0$ vale \$ $(1 + r)$ en $t = 1$).



- Podemos calcular el valor presente:

$$? = \frac{p \times v_u + (1-p) \times v_d}{1+r}$$

...pero ese valor implica riesgo para el vendedor de la opción.

Alternativa: Portafolio de Hedging

- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .

Alternativa: Portafolio de Hedging

- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .
- Entonces, necesitamos:

$$a(1 + u) + (1 + r) = v_u$$

$$a(1 + d) + (1 + r) = v_d$$

Alternativa: Portafolio de Hedging

- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .
- Entonces, necesitamos:

$$a(1 + u) + (1 + r)b = v_u$$

$$a(1 + d) + (1 + r)b = v_d$$

- Resolviendo, tenemos $a = \frac{v_u - v_d}{u - d}$, $b = \frac{v_d(1 + u) - v_u(1 + d)}{(1 + r)(1 + d)}$

Alternativa: Portafolio de Hedging

- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .
- Entonces, necesitamos:

$$a(1+u) + (1+r)b = v_u$$

$$a(1+d) + (1+r)b = v_d$$

- Resolviendo, tenemos $a = \frac{v_u - v_d}{u - d}$, $b = \frac{v_d(1+u) - v_u(1+d)}{(1+r)(1+d)}$
- Ese portafolio requiere una inversión en $t = 0$ de

$$\begin{aligned} a + b &= \frac{v_u(r-d) + v_d(u-r)}{(1+r)(u-d)} \\ &= \frac{1}{1+r} [p_n v_u + (1 - p_n) v_d] \end{aligned}$$

Alternativa: Portafolio de Hedging

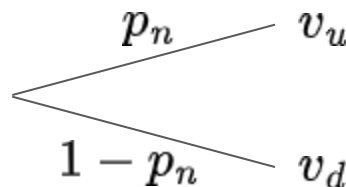
- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .
- Entonces, necesitamos:

$$a(1+u) + (1+r)b = v_u$$

$$a(1+d) + (1+r)b = v_d$$

- Resolviendo, tenemos $a = \frac{v_u - v_d}{u - d}$, $b = \frac{v_d(1+u) - v_u(1+d)}{(1+r)(1+d)}$
- Ese portafolio requiere una inversión en $t = 0$ de

$$\begin{aligned} a + b &= \frac{v_u(r-d) + v_d(u-r)}{(1+r)(u-d)} \\ &= \frac{1}{1+r} [p_n v_u + (1 - p_n) v_d] \end{aligned}$$



Alternativa: Portafolio de Hedging

- Consideramos un portafolio con a unidades del activo y b en la cuenta de ahorros, para replicar las recompensas v_u y v_d .
- Entonces, necesitamos:

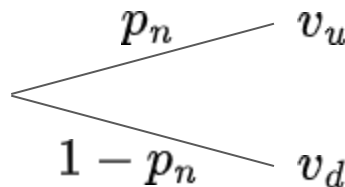
$$a(1+u) + (1+r)b = v_u$$

$$a(1+d) + (1+r)b = v_d$$

- Resolviendo, tenemos $a = \frac{v_u - v_d}{u - d}$, $b = \frac{v_d(1+u) - v_u(1+d)}{(1+r)(1+d)}$

- Ese portafolio requiere una inversión en $t = 0$ de

$$\begin{aligned} a + b &= \frac{v_u(r-d) + v_d(u-r)}{(1+r)(u-d)} \\ &= \frac{1}{1+r} [p_n v_u + (1 - p_n) v_d] \end{aligned}$$



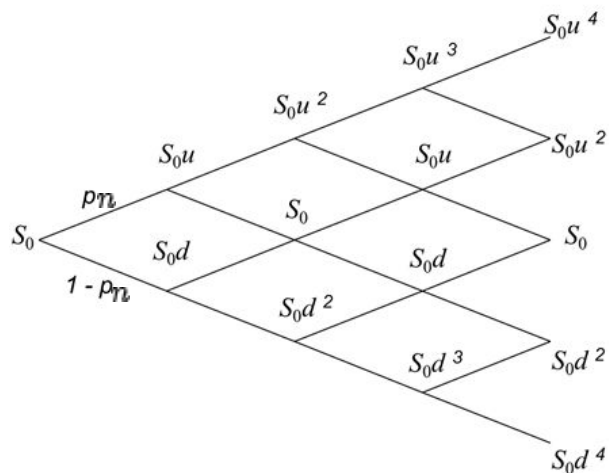
- p_n = probabilidad neutral al riesgo

Probabilidad Neutral de Riesgo

- p_n = probabilidad neutral al riesgo
 - permite evaluar cualquier derivado utilizando el valor presente de las recompensas, con descuento determinado por la tasa de interés sin riesgo
 - no depende de las probabilidades naturales de que el activo suba o baje, solo de su volatilidad
 - $0 \leq p_n \leq p_1$ requiere $d < r < u$, lo que es equivalente a la ausencia de arbitragem
- El portafolio de hedging y la probabilidad neutral al riesgo son conceptos fundamentales en la teoría de opciones.

Modelo Binomial Para Valuación de Opciones

- Estrategia popular en la práctica (Cox-Ross-Rubinstein)
- Se puede construir un árbol binomial con las diferentes posibilidades
- Evaluar por inducción con programación dinámica
- Se vuelve intratable dependiendo del número de períodos y complejidad del modelo
- Alternativa: aprendizaje por refuerzo con simulación Monte Carlo



Modelo Para s_t

- Cómo modelar la evolución de precios del activo?

Modelo Para s_t

- Cómo modelar la evolución de precios del activo?
- Por ejemplo, retornos podrían ser distribuidos normalmente:

$$s_{k+1} = s_k(\mu\Delta t + \sigma\sqrt{\Delta t}\epsilon)$$

Modelo Para s_t

- Cómo modelar la evolución de precios del activo?
- Por ejemplo, retornos podrían ser distribuidos normalmente:

$$s_{k+1} = s_k(\mu\Delta t + \sigma\sqrt{\Delta t}\epsilon)$$

- μ es el retorno esperado del activo
- σ es la volatilidad del activo
- Δt es la duración de cada periodo
 - por ejemplo, si el retorno y volatilidad son anuales y cada periodo corresponde a un día, entonces $\Delta t = \frac{1}{365}$
- ϵ es una variable aleatoria normal (0,1)

Modelo Para s_t

- pero comunmente modelado como lognormal. En versión a tiempo discreto:

$$\log s_{k+1} = \log s_k + (\mu - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}\epsilon$$

Modelo Para s_t

- pero comunmente modelado como lognormal. En versión a tiempo discreto:

$$\log s_{k+1} = \log s_k + (\mu - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}\epsilon$$

- utilizar la probabilidad neutral de riesgo corresponde a sustituir μ por r , la tasa de interés sin riesgo
- exponenciando la ecuación arriba, obtenemos:

$$s_{k+1} = s_k e^{(r-\sigma^2/2)\Delta t + \sigma\sqrt{(\Delta t)}\epsilon}$$

Modelo MDP para Valuación de Opciones

- Debemos definir (S, A, r, P)
- Estado s_t = precio del activo en el periodo t
- Acción $a_t = 0$ (esperar) o 1 (ejercer la opción)
- Recompensa $r_a(s)$:

$$r_0(s) = 0$$

$$r_1(s) = \max(K - s, 0)$$

- Transición P definida implícitamente por la ecuación

$$s_{k+1} = s_k e^{(r - \sigma^2/2)/\Delta t + \sigma\sqrt{(\Delta t)}\epsilon}$$

- El factor de descuento es $\alpha = e^{-r\Delta t}$

Ecuaciones de Bellman

- Cómo definir la función valor? Analizaremos la función Q^*

Ecuaciones de Bellman

- Cómo definir la función valor? Analizaremos la función Q^*
- El valor de ejercer la acción en cualquier periodo t es conocido:

$$Q_t^*(s, 1) = \max(K - s, 0) \equiv (K - s)^+$$

Ecuaciones de Bellman

- Cómo definir la función valor? Analizaremos la función Q^*
- El valor de ejercer la acción en cualquier periodo t es conocido:

$$Q_t^*(s, 1) = \max(K - s, 0) \equiv (K - s)^+$$

- Podemos escribir las ecuaciones para el valor de esperar ($a = 0$):

$$Q_t^*(s, 0) = \alpha \sum_{s'} P(s, s') \max_a Q_{t+1}^*(s', a)$$

Ecuaciones de Bellman

- Cómo definir la función valor? Analizaremos la función Q^*
- El valor de ejercer la acción en cualquier periodo t es conocido:

$$Q_t^*(s, 1) = \max(K - s, 0) \equiv (K - s)^+$$

- Podemos escribir las ecuaciones para el valor de esperar ($a = 0$):

$$\begin{aligned} Q_t^*(s, 0) &= \alpha \sum_{s'} P(s, s') \max_a Q_{t+1}^*(s', a) \\ &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \end{aligned}$$

Ecuaciones de Bellman

- Cómo definir la función valor? Analizaremos la función Q^*
- El valor de ejercer la acción en cualquier periodo t es conocido:

$$Q_t^*(s, 1) = \max(K - s, 0) \equiv (K - s)^+$$

- Podemos escribir las ecuaciones para el valor de esperar ($a = 0$):

$$\begin{aligned} Q_t^*(s, 0) &= \alpha \sum_{s'} P(s, s') \max_a Q_{t+1}^*(s', a) \\ &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \end{aligned}$$

- En el último periodo, ejercitamos si $K - s > 0$, si no la opción no vale nada:

$$Q_T^*(s, \cdot) = (K - s)^+$$

Problemas de Parada Óptima

- es suficiente calcular o aprender $Q_t^*(\cdot, 0)$. Las ecuaciones de Bellman pueden ser escritas solamente con esa función:

$$\begin{aligned}Q_t^*(s, 0) &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \\ Q_{T-1}^* &= \alpha \sum_{s'} P(s, s') (K - s')^+\end{aligned}$$

Problemas de Parada Óptima

- es suficiente calcular o aprender $Q_t^*(., 0)$. Las ecuaciones de Bellman pueden ser escritas solamente con esa función:

$$\begin{aligned}Q_t^*(s, 0) &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \\ Q_{T-1}^* &= \alpha \sum_{s'} P(s, s') (K - s')^+\end{aligned}$$

- esa clase de MDPs es conocida como problemas de parada óptima

Problemas de Parada Óptima

- es suficiente calcular o aprender $Q_t^*(\cdot, 0)$. Las ecuaciones de Bellman pueden ser escritas solamente con esa función:

$$\begin{aligned}Q_t^*(s, 0) &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \\ Q_{T-1}^* &= \alpha \sum_{s'} P(s, s') (K - s')^+\end{aligned}$$

- esa clase de MDPs es conocida como problemas de parada óptima
- resolviendo con aprendizaje por refuerzo:
 - generamos trayectorias $(s_0^i, s_1^i, s_2^i, \dots)$, $i = 1, \dots, N$ siguiendo

$$s_{k+1} = s_k e^{(r - \sigma^2/2)/\Delta t + \sigma\sqrt{(\Delta t)}\epsilon}$$

Problemas de Parada Óptima

- es suficiente calcular o aprender $Q_t^*(\cdot, 0)$. Las ecuaciones de Bellman pueden ser escritas solamente con esa función:

$$\begin{aligned}Q_t^*(s, 0) &= \alpha \sum_{s'} \max((K - s')^+, Q_{t+1}^*(s', 0)) \\ Q_{T-1}^* &= \alpha \sum_{s'} P(s, s') (K - s')^+\end{aligned}$$

- esa clase de MDPs es conocida como problemas de parada óptima
- resolviendo con aprendizaje por refuerzo:
 - generamos trayectorias $(s_0^i, s_1^i, s_2^i, \dots)$, $i = 1, \dots, N$ siguiendo

$$s_{k+1} = s_k e^{(r - \sigma^2/2)/\Delta t + \sigma\sqrt{(\Delta t)}\epsilon}$$

- aplicamos métodos de aprendizaje por refuerzo para resolver las ecuaciones de Bellman (simulación Monte Carlo, TD-Learning, versiones de iteración de valor e iteración de políticas,...)

Complejidad del MDP

- con el modelo que estamos utilizando para el precio del activo s_t tenemos solo una variable de estado/dimensión
- en la práctica, puede ser necesario incorporar muchas otras variables de estado:
 - volatilidad σ y/o tasa de interés r pueden ser procesos que varían en el tiempo
 - pueden depender de otros factores
 - activo puede pagar dividendos
 - opción puede ser mucho más compleja y recompensa puede estar basada en distintas condiciones

Complejidad del MDP

- con el modelo que estamos utilizando para el precio del activo s_t tenemos solo una variable de estado/dimensión
- en la práctica, puede ser necesario incorporar muchas otras variables de estado:
 - volatilidad σ y/o tasa de interés r pueden ser procesos que varían en el tiempo
 - pueden depender de otros factores
 - activo puede pagar dividendos
 - opción puede ser mucho más compleja y recompensa puede estar basada en distintas condiciones
- nuestro modelo asume que precios tienen valores continuos
 - número infinito de estados

Aproximaciones para $Q_t^*(., 0)$

- mismo en el caso más simple de una variable de estado, aproximaciones son necesarias
- estrategia popular en la práctica (Longstaff y Schwartz):
 - basada en simulación Monte Carlo y inducción reversa
 - aproximación con arquitectura linear

$$Q_t^*(s, 0) \approx \sum_{k=0}^N \phi^i(s) w^i$$

- por ejemplo, $\phi^i(s) = s^i$ (aproximación polinomial)

Aproximación con Simulación Monte Carlo

1. Generamos M trayectorias $(s_0^i, s_1^i, \dots, s_T^i)$, $i = 1, \dots, M$
2. En $t = T - 1$, resolvemos

$$w_{T-1} = \arg \min_w \sum_{i=1}^M [\Phi(s_{T-1}^i, w) - \alpha(K - s_T^i)^+]^2$$

3. Para $t = T - 2, T - 3, \dots, 0$ resolvemos

$$w_t = \arg \min_w \sum_{i=1}^M [\Phi(s_t^i, w) - \alpha \max((K - s_{t+1}^i)^+, \Phi(s_{t+1}^i, w_{t+1}))]^2$$

donde $\Phi(s, w) = \sum_{j=0}^N \phi^j(s) w^j$ y $\alpha = e^{-r\Delta t}$

Aproximación con Simulación Monte Carlo

1. Generamos M trayectorias $(s_0^i, s_1^i, \dots, s_T^i)$, $i = 1, \dots, M$
2. En $t = T - 1$, resolvemos

$$w_{T-1} = \arg \min_w \sum_{i=1}^M [\Phi(s_{T-1}^i, w) - \alpha(K - s_T^i)^+]^2$$

3. Para $t = T - 2, T - 3, \dots, 0$ resolvemos

$$w_t = \arg \min_w \sum_{i=1}^M [\Phi(s_t^i, w) - \alpha \max((K - s_{t+1}^i)^+, \Phi(s_{t+1}^i, w_{t+1}))]^2$$

donde $\Phi(s, w) = \sum_{j=0}^N \phi^j(s) w^j$ y $\alpha = e^{-r\Delta t}$

- Inducción reversa
- Las minimizaciones corresponden a regresión lineal
- Implementación en Python

Conclusiones

- Paradigma de MDP es poderoso y suficientemente flexible para modelar una amplia variedad de aplicaciones en distintos campos
- Combinada con aprendizaje, potencial para encontrar soluciones competitivas
- Aproximaciones a la función valor y/o política permite tratar problemas de alta complejidad y muchas dimensiones
 - Desafíos en la implementación estable y correcta de los algoritmos
 - Complejidad del abordaje debe ser acorde a los datos disponibles
 - Potencialmente grandes beneficios con la adopción de redes neuronales